Practical 1(a): Gender Bias Detection

Aim: To detect gender bias by comparing the approval rates of males and females.

```
data = [[1,1],[1,1],[1,1],[0,0],[0,1],[0,0]]

m = [x[1] for x in data if x[0] == 1]
f = [x[1] for x in data if x[0] == 0]

mr = sum(m) / len(m)
fr = sum(f) / len(f)
gap = abs(mr - fr)

print(f"Male approval rate: {mr:.0%}")
print(f"Female approval rate: {fr:.0%}")

print(f"\nBias gap: {gap:.0%}")

print("Significant Bias detected" if gap > 0.15
      else "Within acceptable range")
```

Practical 1(b): Fairness Assessment Using the 80% Rule

Aim: To evaluate whether a minority group receives approvals at a sufficiently comparable rate to a majority group.

```
p = {
   "majority": {"approved": 80, "denied": 20},
   "minority": {"approved": 45, "denied": 55}
}

def rate(g):
   return g["approved"] / (g["approved"] + g["denied"])

maj = rate(p["majority"])
minr = rate(p["minority"])
ratio = minr / maj

print(f"Majority rate: {maj:.0%}")
print(f"Minority rate: {minr:.0%}")
print(f"Impact ratio: {ratio:.2f}")

if ratio < 0.8:
   print("Bias detected")
else:
   print("Passes 80% rule")
```

Practical 2(a): Personally Identifiable Information (PII) Detection

Aim: To identify sensitive personal information in a dataset using regular expressions.

```
import re

data = [
    "John Doe",
    "alice@example.com",
    "9876543210",
    "123-45-6789"
]

patterns = [
    r"\b[A-Z][a-z]+ [A-Z][a-z]+\b",
    r"\S+@\S+\.\S+",
    r"\b\d{10}\b",
    r"\d{3}-\d{2}-\d{4}"
]

found = False

for i, p in enumerate(patterns):
    if any(re.search(p, x) for x in data):
        print(["Name", "Email", "Phone", "SSN"][i], "detected")
        found = True

print("Anonymise data" if found else "No PII detected")
```

Practical 2(b): Identification of Protected Attributes

Aim: To identify protected attributes among the features used by an AI model.

```
features = {
    "income": 0.72,
    "credit history": 0.65,
    "age": 0.41,
    "zip-code": 0.38,
    "employment": 0.30,
    "gender": 0.27,
    "debt-ratio": 0.20
}

protected = {"gender", "age", "zip-code"}

for f, w in sorted(features.items(), key=lambda x: -x[1]):
```

```
    print(f, w, "PROTECTED" if f in protected else "")

print("Flagged:", protected)
```

Practical 3: Explainable AI — Loan Decision Explanation

Aim: To explain an AI-based loan decision by examining the contributions of individual features to the final score

```
a = {
    "income": 0.68,
    "credit_score": 0.55,
    "loan_amount": -0.47,
    "employment_yrs": 0.30,
    "age": 0.12,
    "has_default": 0.00
}

print("Loan Decision Explanation")
print("-" * 35)
print(f"Raw score: {sum(a.values()):+.2f}\n")

for f, w in sorted(a.items(), key=lambda x: -abs(x[1])):
    print(f"{f:18} {w:+.2f} "
          f"{'APPROVE' if w > 0 else 'REJECT'}")

print("\nFinal decision:",
      "APPROVED" if sum(a.values()) > 0 else "REJECTED")
```

Practical 4(a): AI Confidence Calibration

Aim: To compare the confidence claimed by an AI model with its actual prediction accuracy across different confidence ranges.

```
p = [
    (0.97,1),(0.95,1),(0.92,0),(0.91,1),
    (0.88,0),(0.85,1),(0.84,0),(0.80,1),
    (0.78,0),(0.75,1),(0.60,1),(0.55,0)
]

bands = {
    "90-100%": [],
    "75-89%": [],
    "50-74%": []
}
```

```
for c, y in p:
    bands["90-100%" if c >= .9 else
          "75-89%" if c >= .75 else
          "50-74%"].append(y)

for b, v in bands.items():
    actual = sum(v) / len(v)
    claimed = {
        "90-100%": .94,
        "75-89%": .82,
        "50-74%": .57
    }[b]

    print(b, f"Claimed:{claimed:.0%} Actual:{actual:.0%}",
          "OK" if abs(claimed - actual) < .1
          else "OVERCONFIDENT")
```

Practical 4(b): Data Privacy and Consent Audit

Aim: To audit data fields for missing consent, excessive retention, and sensitive information without consent

```
data = [
    ("name",1,365),
    ("email",1,365),
    ("location_gps",0,730),
    ("health_record",0,999),
    ("browsing_hist",1,180),
    ("face_images",0,999),
    ("age",1,90)
]

sensitive = {"location_gps", "health_record", "face_images"}

print(f"{'Field':15} {'Consent':8} {'Retention':10} Status")

n = 0
```

```
for f, c, d in data:
    issues = (
        ["no consent"] if not c else []
    ) + (
        ["kept too long"] if d > 365 else []
    ) + (
        ["SENSITIVE"] if f in sensitive and not c else []
    )

    n += bool(issues)

    print(f"{f:15} {bool(c)!s:8} {d:<10} "
          f"{', '.join(issues) or 'OK'}")

print(f"\nViolations found: {n}/{len(data)}")
```

Practical 5: Sentiment Analysis Robustness Testing

Aim: To test a simple sentiment analysis function against positive, negative, and neutral statements, including negation and punctuation.

```
def sentiment(t):
    pos = ["good", "great", "happy", "excellent", "safe", "love"]
    neg = ["bad", "hate", "unsafe", "terrible", "dangerous"]

    t = t.lower()
    s = sum(w in t for w in pos) - sum(w in t for w in neg)

    return "POSITIVE" if s > 0 else "NEGATIVE" if s < 0 else "NEUTRAL"
```

```
tests = [
    ("This drug is safe", "POSITIVE"),
    ("This drug is safe", "POSITIVE"),
    ("This drug is not safe", "NEGATIVE"),
    ("This drug is safe!!!", "POSITIVE"),
    ("SAFE drug excellent care", "POSITIVE"),
    ("This drug is terrible", "NEGATIVE"),
    ("This drug isn't terrible", "POSITIVE")
]

n = 0

print(f"{'Input':30} {'Expected':10} {'Got':10} Pass?")

for t, e in tests:
    r = sentiment(t)
    ok = r == e
    n += ok

    print(f"{t:30} {e:10} {r:10} "
          f"{'PASS' if ok else 'FAIL'}")

print(f"\nRobustness score: {n}/7 ({n/7:.0%})")
```

Practical 6: Group Fairness and Disparity Detection

Aim: To compare approval rates between two groups while excluding records marked with the third field as `1`.

```
records = [
```

```
    ("A",1,0),("A",1,1),("A",0,0),
    ("A",1,0),("A",0,0),("A",1,1),
    ("B",1,0),("B",1,0),("B",0,0),
    ("B",1,1),("B",1,0),("B",0,0)
]

rates = {}

for g in ["A", "B"]:
    x = [r for r in records if r[0] == g and not r[2]]
    rates[g] = sum(r[1] for r in x) / len(x)
    print(f"Group {g}: {rates[g]:.0%}")

gap = abs(rates["A"] - rates["B"])

print(f"Gap between groups: {gap:.0%}")
print("VERDICT:",
      "Disparity detected" if gap > 0.1
      else "Comparable rates")
```

Practical 7(a): Explainable AI Using SHAP Values

Aim: To explain an AI model's prediction by showing how individual features increase or reduce the predicted risk.

```
s = {
    "missed_payments": .42,
    "credit_utilisation": .31,
    "income": -.38,
    "loan_term": .18,
    "employment_yrs": -.22,
```

```
    "num_accounts": -.09,
    "recent_enquiries": .14
}

b = .20
p = b + sum(s.values())

print("SHAP Explanation: Applicant #4821")
print(f"Base value: {b:.2f}")
print(f"Sum of SHAP values: {sum(s.values()):+.2f}")
print(f"Final prediction: {p:.2f}")

for f, v in sorted(s.items(), key=lambda x: -abs(x[1])):
    print(f"{f:20} {v:+.2f} "
          f"{'INCREASES risk' if v > 0 else 'REDUCES risk'}")

print(
    f"VERDICT: DEFAULT RISK (score = {p:.2f})"
    if p > .5 else
    f"VERDICT: LOW RISK (score = {p:.2f})"
)
```

Practical 7(b): Local Feature Importance Using Perturbation

Aim: To identify the most influential feature by changing each input by 10% and observing the change in the model prediction.

```
def predict(f):
    w = {
        "age": .1,
        "income": -.4,
```

```
        "debt": .5,
        "credit_score": -.45,
        "employed": -3
    }
    return sum(f[k] * w[k] for k in w)


o = {
    "age": 45,
    "income": 62000,
    "debt": 18000,
    "credit_score": 680,
    "employed": 1
}

base = predict(o)
impact = {}

print(f"Original prediction: {base:.3f}")
print("Feature       Original Perturbed Impact")

for k, v in o.items():
    n = o.copy()
    n[k] = v * 1.1
    d = predict(n) - base
    impact[k] = d

    print(f"{k:14} {v:<9} {n[k]:<10.0f} {d:+.3f}")

print("Most influential feature locally:",
      max(impact, key=lambda k: abs(impact[k])))
```

Practical 8: Counterfactual Explanations for Loan Rejection

Aim: To identify actionable changes in an applicant's profile that may help improve their loan eligibility.

```
applicant = {
    "credit_score": 580,
    "income": 32000,
    "missed_payments": 3,
    "employment_yrs": 1,
    "loan_amount": 15000
}

risk = 0.73
threshold = 0.50

changes = {
    "credit_score": 650,
    "missed_payments": 1,
    "income": 45000,
    "employment_yrs": 2
}

print(f"Current risk: {risk} -> REJECTED")
print("Feature       Old    New    Feasible")

for k, v in changes.items():
    print(f"{k:16} {applicant[k]:<7} {v:<7} Yes")

print("Best actionable change: missed_payments from 3 to 1")
```

Practical 9: Data Anonymisation in Healthcare

Aim: To anonymise patient records by removing direct identifiers and converting exact ages into age bands.

```
patients = [
    {"name":"Alice Martin","age":34,"email":"alice@mail.com",
     "diagnosis":"hypertension"},
    {"name":"Bob Singh","age":52,"email":"bob@mail.com",
     "diagnosis":"diabetes"},
    {"name":"Carol Osei","age":28,"email":"carol@mail.com",
     "diagnosis":"asthma"},
    {"name":"David Kim","age":61,"email":"david@mail.com",
     "diagnosis":"arthritis"}
]

print("BEFORE anonymisation:")
for p in patients:
    print(p)

for p in patients:
    p["name"] = p["email"] = "REDACTED"
    p["age_band"] = f"{p['age']//10*10}s"
    del p["age"]

print("\nAFTER anonymisation:")
for p in patients:
    print(p)

print("Direct identifiers remaining: 0/4")
print("STATUS: Identifiers removed")
```

Practical 10: Data Lineage Tracking in AI Systems

Aim: To track the source, ownership, transformations, and personally identifiable information (PII) status of data throughout an AI pipeline.

```
pipeline = [
    ("Raw Ingestion", "Hospital EHR API", "Data Eng",
     ["ingest"], True),
    ("PII Removal", "Raw Ingestion", "Privacy Team",
     ["drop PII"], False),
    ("Normalization", "PII Removal", "Data Eng",
     ["standardize"], False),
    ("Model Training", "Normalization", "ML Team",
     ["train/test split"], False)
]

print("DATA LINEAGE REPORT")

for i, (step, src, owner, trans, pii) in enumerate(pipeline, 1):
    print(
        f"{i}. {step} | Owner: {owner} | Source: {src} "
        f"| PII: {pii} | Transform: {', '.join(trans)}"
    )
```